COMP 4311 Programming Languages
Homework 1

1. Consider the following pseudocode, assuming nested subroutines and static scope:

```
procedure main()
    g : integer
    procedure B(a : integer)
        x : integer
        procedure A(n : integer)
            g := n
        procedure R(m : integer)
            write integer(x)
            x := x / 2
            if x > 1
                R(m + 1)
            else
                A(m)
        – – body of B
        x := a + a
        R(1)
    – – body of main
    B(3)
    write integer(g)
```

(a) What does this program print, show detailed steps. Hints: you can draw calling stacks, 5/2 = 2 for integer division (flooring) (1.5pts)

(b) Explain how A finds g. (1pt)

(a) Step-by-step execution

Step 1: Enter main

Variable:

g (uninitialized yet)

Step 2: Call B(3)

Create new activation record for B:

parameter a = 3

local x

Inside B we also have nested procedures A and R.

Step 3: Execute body of B

x := a + a = 3 + 3 = 6

Now call R(1).

Step 4: R(1) executes

Local variable x belongs to B, so R sees the same x.

write integer(x) ⟶ prints 6

x := x / 2 ⟶ integer division ⟶ x = 6 / 2 = 3

Condition: x > 1 ⟶ 3 > 1 ⟶ true⟶ Call R(m + 1) ⟶ R(2)

Step 5: R(2)

write integer(x) ⟶ prints 3

x := x / 2 ⟶ 3 / 2 = 1

x > 1 ? ⟶ false ⟶ execute A(m) with m = 2.

Step 6: Call A(2)

A assigns g := n = 2.

Return to R(2) ⟶ return to R(1) ⟶ return to B.

Step 7: Back in B

R finished; x is 1 now, but that doesn't matter anymore.

Return from B to main.

Step 8: Back in main

Execute write integer(g) ⟶ prints 2.

We printed:

6

3

2

(b) How A finds g

Because static scope (lexical scope) is assumed:

Procedure A is nested inside B, but B is nested inside main.

When A executes and refers to g, it looks for a declaration of g in its lexical enclosing scopes.

A does not declare g, nor does B.

It finds g in the outermost static scope, which is main.

So A statically links to main's activation record, and g refers to that variable.

2. Consider the following fragment of code in C:

```
{   int a, b, c;
    ...
    {    int d, e;
         ...
         {    int f;
              ...
         }
         ...
    }
    ...
    {   int g, h, i;
        ... }
```

...

}

(a) Assume that each integer variable occupies four bytes. How much total space is required for the variables in this code (assume we use static memory allocation only)? (1pt)

(b) Describe how a compiler could use to assign stack frame offsets to the variables of arbitrary nested blocks, in a way that minimizes the total space required, what is the minimum required memory (2pt)

ANS:

a) 9*4=36 bytes
b) If we use stack, we need 6*4=24 bytes only.

3. Consider the following pseudocode:

```
x: integer := 2
y: integer := 4
procedure add
    x:= x+y
    write_integer(x)
procedure third (P: procedure)
    x: integer := 8
    p()
procedure second()
    y: integer := 6
    third(add)
procedure first()
    x: integer := 10
    y : integer:= 16
    second
first()
```

(a) What does this program print if the language uses static scoping? (1.5pt)

(b) What does it print if the language uses dynamic scoping with deep binding? (1.5pt)

(c) What does it print if the language uses dynamic scoping with shallow binding? (1.5pt)

Hints: brief descriptions or detailed steps required.

ANS:

a) Add defined under main, use global x and y, 2+4=6
b) Add first called (defined) from second, use second's x and y, second has the local y and x from first, 6+10 = 16.
c) Add called from third, use third's x and y, third has loca x and y from second: 8+6=14